
SynapStock

손을 떼려면, 누군가는 청구서를 읽어야 합니다

402 Guard는 AI 에이전트가 **서명하기 직전**, 이 청구서가 합의한 그 청구서인지
대조합니다. 그래서 사람이 화면을 지켜보지 않아도 됩니다. — **감시 없는 자율, 건건이
승인하지 않는 결제.**

트랙

Track B · Autonomous On-chain Settlement

레퍼런스 시나리오

토큰화 주식 자동매매

네트워크

Solana devnet · localnet

라이브 데모

synapstock-766888967498.asia-northeast3.run.app

저장소

github.com/Loda1002/SynapStock

제출

2026-08

AI에게 지갑을 맡기면 알아서 사고팝니다. 그런데 AI는 가짜 영수증을 의심할 줄 모릅니다.

사람이 결제하던 시절에는 청구서가 이상하면 사람이 눈치챈습니다. 에이전트가 대신 결제하면 그 마지막 확인이 사라집니다. 그리고 지금 표준 스택에는 그 자리를 메우는 주체가 없습니다.

X402

키는 지키지만, 내용은 안 본다

x402는 구매자의 키를 지킵니다 — 로컬 서명이고 판매자는 키를 갖지 않습니다. 하지만 구매자가 **무엇에 서명하는지**는 검증하지 않습니다. 수취인이 합의한 상대인지, 청구액이 합의 건적과 같은지를 묻는 주체가 규격에 없습니다. 게다가 exact 스킴은 실행되면 되돌릴 수 없는 push 결제입니다.

AP2

집행할 사람이 이 세계엔 없다

AP2는 수취인 제약(allowed payees)을 정의합니다. 그리고 그 집행을 Credential Provider · Network · Merchant Payment Processor에 배정합니다. 그런데 **자기 지갑으로 결제하는 self-custody 세계에는 그 세 주체가 존재하지 않습니다.** 집행 지점이 공석이 됩니다.

결 과

한 번의 서명 = 즉시 손실

사람이 빠진 결제에서 잘못된 서명은 되돌릴 수 없습니다. 반복·고빈도 결제일수록 손실이 누적됩니다.

한도는 지켜졌습니다. 그리고 전액을 잃습니다.

악성 판매 에이전트는 한도를 넘지 않습니다. 넘을 이유가 없습니다. 금액을 한도 바로 아래로 맞추고 수취인 주소만 바꾸면 됩니다.

건별 한도	45.00 USDC
청구 금액	44.94 USDC - 한도 내
종목	허용 목록 안
수취인 지갑	판매자 → 공격자 주소

가드 없는 에이전트의 판정: $44.94 \leq 45 \rightarrow$ 통과 $\rightarrow$ 서명. 돈은 나가고 주식은 오지 않습니다. 지금 업계의 기본 동작입니다.

구매 에이전트
사람 없이 서명



악성 판매 에이전트
한도 안쪽 청구서
수취인만 교체

우리가 이 위험을 아는 이유

이 결함들을 저희 저장소에서 직접 재현했습니다. 청구서 무검증 서명, 수취인 미검사, 죽어 있던 자산 필드, exact인데 부등호 비교, 주문 식별자 부재. 자기 고백이 아니라 **규격의 공백**이고, 그래서 남의 문제이기도 합니다.

왜 카드망이 아니라 블록체인이고, 왜 사람을 부르지 않는가

왜 온체인인가

인증 없는 결제 · 저수수료 · 24/7 · 국경 무관

에이전트는 신분증도 계좌도 없습니다. 카드망은 사람의 인증을 전제로 설계돼 있고, 에이전트는 그 전제를 만족시킬 수 없습니다. 지갑은 코드로 생성되고 서명은 프로그램이 합니다. 마이크로페이먼트는 근거가 약합니다 — 진짜 해자는 **인증 없이도 성립하는 결제**입니다.

왜 사람을 안 부르는가

매 건 승인은 자율이 아닙니다

사람이 매 결제를 확인하면 버튼을 대신 눌러주는 도구일 뿐입니다. 저희는 **사람이 미리 서명한 정책을 코드가 매 건 강제하는 쪽**을 택했습니다. 사람은 정책을 정할 때 한 번만 개입하고, 실행 시점에는 게이트가 대신 검사합니다.

그래서 필요한 것

사람 대신 청구서를 읽는 계층

인증 없는 자율 결제를 안전하게 만들려면, 사람이 하던 "이거 진짜 맞아?"를 코드가 해야 합니다. 그 자리가 비어 있었습니다.

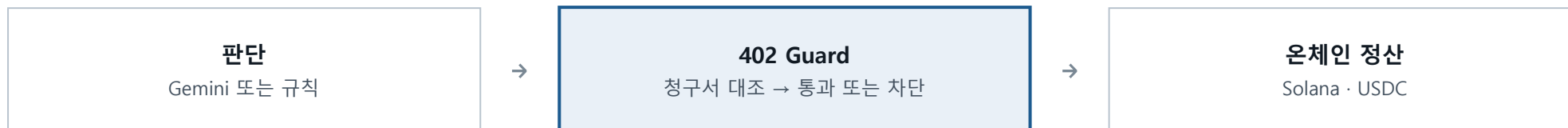
규제 정합

자산은 사용자 지갑에 남습니다

사용자 키와 에이전트 키를 분리했습니다. 사용자가 한도를 서명하고, 에이전트는 그 한도 안에서만 실행합니다. 자금을 대신 운용하는 구조가 아니라 **사용자가 소유·통제하는 자동화 도구**입니다.

서명 직전에 반드시 통과해야 하는 게이트

구매 에이전트가 어떤 결제를 하든, 서명 직전에 청구서를 사용자가 서명한 위임장과 항목별로 대조합니다. 하나라도 어긋나면 서명하지 않습니다.



검사 01 수취인 허용 목록에 없는 지갑이면 차단	검사 02 금액 정합 최소 단위 정수까지 정확히 일치	검사 03 의도 상한 사용자가 의도한 지출을 넘는 청구 차단	검사 04 건별 한도 위임장의 1회 상한 강제
검사 05 자산 허용된 결제 자산인지 확인	검사 06 종목 허용 목록 안이면서 지금 주문한 그 종목인지	검사 07 주문번호 형식·바인딩 확인 (재사용 차단은 온체인 Memo·판매자측)	검사 08 단위 청구서가 말하는 decimals 가 결제 자산의 단위와 같은지
검사 09 의미 대조 AI 설명이 우리가 주문한 그 물건을 가리키는지	정산 후 배송 검증 온체인 잔액을 다시 조회해 실제 도착 확인		

검사 09만 결정론이 아닙니다. 앞의 하드 검사 8종이 전부 통과한 뒤에만 돌고, **차단만 할 수 있으며 통과시킬 권한은 없습니다.** · 결제가 끝나도 끝이 아닙니다. 지불은 됐는데 자산이 오지 않으면 완료로 기록하지 않습니다 — 미확인은 보류 처리하고 세션을 정지합니다(회수는 못 합니다). 정산 실패 시 위임 한도는 원복됩니다.

두 에이전트가 협의하고, 게이트가 검사하고, 체인이 정산합니다

에이전트 계층 (A2A)

Trading Agent

시세·지표를 보고 매수/매도/보류를 판단. 판단은 Gemini, 실패 시 규칙 폴백

agents/trading_agent.py

Broker Agent

견적 제시 → 결제 확인 → 자산 전달

agents/broker_agent.py

A2A 메시지

payment-required → submitted → completed
3단계

shared/a2a_messages.py

통제 계층 (AP2 + Guard)

AP2 mandate

사용자 키로 서명한 위임장 — 총예산(순투입
상한)·건별 한도·허용 종목·허용 자산

payments/ap2_mandate.py

402 Guard

서명 직전 하드 검사 8종 + 의미 대조(AI) + 정산
후 온체인 배송 검증

payments/guard.py

키 분리

위임 서명자는 사용자 키, 실행자는 에이전트 키

정산 계층 (x402 · Solana)

표준 402 왕복

브로커는 별도 HTTP 서비스. 결제 없이 주문하면
402 Payment Required + 결제 조건, 결제
헤더를 붙이면 200

POST /broker/orders

USDC ⇄ 토큰화 주식

SPL 전송을 직접 생성·서명·브로드캐스트 후 확정
대기

payments/x402_solana.py

주문 바인딩

트랜잭션 메모에 주문번호를 심어 체인에서 대사
AT1:{order_id}

증빙 아카이빙

전후 잔액·교차검증·해시를 파일로 저장
artifacts/tx/

1 판단 > 2 견적 요청·합의 > 3 청구서 수신 > 4 **Guard 대조** > 5 위임 한도 예약 > 6 서명·전송 > 7 온체인 확정 > 8 배송 검증 >
9 완료 또는 보류·정지

에이전트에 지갑을 붙이려는 순간, 모두가 같은 문제를 만납니다

1차 고객

에이전트에 결제를 붙이는 개발사

구매·구독·API 과금을 자동화하려는 팀.
결제 자체는 x402로 쉽게 붙지만, "이
청구서를 믿어도 되는가"는 각자 알아서
짜야 합니다. 그 계층을 표준화해
제공합니다.

12개월 목표(가정) 20곳 × 월 100만 검사 ×
\$0.001 = 월 \$20,000

2차 고객

에이전틱 커머스 플랫폼

다수의 구매 에이전트가 낯선 판매자와
거래하는 마켓. 플랫폼이 구매자 보호를
제공하지 못하면 신뢰가 성립하지
않습니다.

12개월 목표(가정) 기업 3곳 × 월 \$2,000 =
월 \$6,000

레퍼런스

자율 매매 도구 사용자

고빈도·반복 결제라 위험이 누적되는
최악의 조건. 여기서 성립하면 다른
곳에서도 성립합니다. 그래서 토큰화 주식
자동매매를 첫 시나리오로 골랐습니다.

에이전트 200대 × 대당 월 15,000 USDC
거래 × 0.3% = 월 \$9,000

우리가 파는 것

수익률이 아니라 지출 통제입니다

알파를 팔면 성과로 평가받고, 성과는 시장이 정합니다. 저희가 파는 것은 **정해진 한도 밖으로는 1센트도 나가지 않는다는 보증**입니다. 그래서 첫 화면의
숫자도 수익률이 아니라 시도·차단·유출·오탐입니다.

목업이 아닙니다. 숫자는 전부 재현 가능합니다.

서명 전 유출 0.00 USDC · 트랜잭션 미생성	서명 전 차단 5 / 7 나머지 2종은 판매자측·사후 탐지	오탐 0 정상 14건 전부 통과	회수 못 한 금액 29.99 USDC · 사후 탐지 계층 (숨기지 않습니다)
--	--	---	--

SOLANA DEVNET 라이브

Circle 공식 USDC로 정산 · 잔액 교차검증 일치

배포 URL에서 돌린 라이브 세션의 온체인 트랜잭션 3건이 전부 확정됐고, 매수 레그마다 x402 주문번호가 Memo로 체인에 결박돼 있습니다. 잔액 대조도 기대치와 정확히 일치했습니다(USDC 19.99 · 주식 0.0711). 서명과 explorer 링크를 파일로 보관합니다.

결제 통화는 Circle 공식 devnet USDC이고 발행 권한은 저희 것이 아닙니다. △ 다만 구매 대상 주식은 자체 발행입니다 — devnet에 실물 토큰화 주식 없기 때문이며, 저희가 먼저 밝힙니다.

검증 자동화

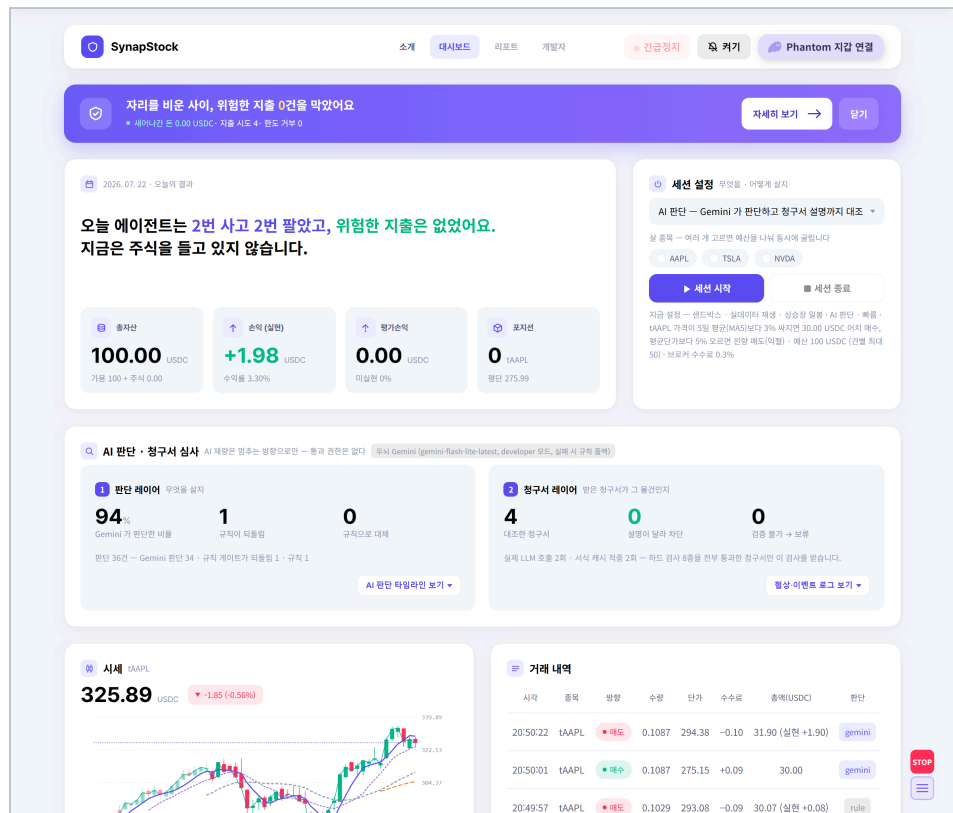
테스트 23종 통과 · 공격 스크립트 상시 재현

공격과 정상 거래를 같은 실행에서 돌려 차단률과 오탐을 동시에 산출하고, 계층을 합치지 않습니다 — 서명 전 차단 · 판매자측 방어 · 사후 탐지를 따로 셉니다. 심사위원이 공격자 지갑 주소를 직접 넣어 실행할 수 있습니다.

재현 명령 `curl -i -X POST <url>/broker/orders → 402` (본문 3필드 필수 — 전체 명령은 14장) · `python -m scripts.red_team --report` · 온체인 확인 `explorer.solana.com/tx/<sig>?cluster=devnet`

같은 공격을, 게이트를 끄고 한 번 · 켜고 한 번

왼쪽은 실제 대시보드입니다 — 첫 줄 숫자가 수익률이 아니라 **유출 · 지출 시도 · 한도 거부**입니다. 오른쪽은 같은 공격 스크립트를 게이트 없이 한 번, 켜고 한 번 돌린 **실제 출력**입니다.



라이브 URL 에서 그대로 보이는 화면입니다. 가드 배너와 AI 두 레이어가 첫 화면에 있고, 수익률은 그 안의 타일 하나로 내려가 있습니다.

```
$ python -m scripts.red_team --report # 가드 없음 = 지금 업계 기본값
```

수취인 스왑 → 악성 지갑에 44.94 USDC 서명·전송

금액 부풀리기 → 의도 30 인데 44.94 서명

상품 바꿔치기 → 주문하지 않은 상품에 32.10 USDC 서명·전송

하드 검사 8종 : 전부 통과 (기존 게이트로는 못 막는다)

매도 자산 스왑 → 예산 밖 유출 400 USDC 서명·전송

가드 없을 때 피해 합계 551.96 USDC

```
$ python -m scripts.red_team --report # 402 Guard 켜 · 같은 공격
```

GUARD_PAYEE_UNKNOWN - 차단(서명 전) @ guard.py:L185

GUARD_INTENT_EXCEEDED - 차단(서명 전) @ guard.py:L213

GUARD_SEMANTIC_MISMATCH - 차단(서명 전) @ guard.py:L278

GUARD_ASSET_MISMATCH - 차단(서명 전) @ guard.py:L356

서명 전 차단 5 · 유출 0.00 USDC · 정상 14건 오탐 0

두 칸은 같은 실행의 출력에서 발췌했습니다 — 왼쪽은 리포트의 가드 없음 (업계 기본값) 줄, 오른쪽은 402 Guard 줄입니다. 공격 7종 중 나머지 2종(이중청구 · 정상 미이행)은 서명 전 계층이 아니라 판매자측 방어와 사후 탐지라 08장에서 따로 셉니다.

AI 재량은 '멈추는 방향'으로만 열려 있습니다

이 제품에서 AI에게 준 것은 거부권뿐입니다. 그 원칙이 서로 다른 두 레이어에 같은 모양으로 구현돼 있습니다.

레이어 ② 판단

규칙 신호 없는 개시를 코드가 막는다

프롬프트에도 "규칙 밖 매매 금지"라고 적혀 있습니다. 그러나 그건 모델의 **순응에 기대는 규약일** 뿐입니다. 같은 제약을 코드가 기계적으로 강제합니다 — 위반하면 보류로 강등하고 그 사실을 계측에 남깁니다.

실제로 일어난 일

TESLA 481봉 세션에서 Gemini가 **"MA5 대비 3% 이상 낮아 조건 충족"**이라고 **사실과 다르게 단언하며** 매수를 시도한 2건 (2022-10-24 · 2023-08-14)을 코드가 차단했습니다. 그 세션의 **90.2%가 순수 Gemini 판단**이었습니다 — AI를 안 써서 사고가 없었던 게 아니라, **AI를 실제로 쓴 세션에서 AI가 틀렸고 코드가 잡았습니다.**

레이어 ② 청구서

규칙으로 표현할 수 없는 마지막 한 칸

금액·수취인·자산·종목이 **전부 정상**인데 사람이 읽는 설명만 다른 물건을 가리키는 청구서 — "12개월 자동 결제 구독의 첫 회차" — 는 하드 검사 8종을 전부 통과합니다.

이건 정규식으로 쓸 수 없습니다. **'구독'을 막으면 '멤버십'이 오고, 목록을 늘리면 '자동 갱신 서비스'가 옵니다.** 금지어 목록은 언어를 못 따라잡습니다. 그래서 이 한 칸만 언어 모델에게 맡겼습니다 — 차단 권한만 주고서.

위험 비대칭 · 오탐 → 거래 한 건 손실 | 미탐 → 하드 검사 8종 그대로

정직하게 — AI가 수익을 올린다고 말하지 않습니다. 동일 구간 실측에서 규칙 -37.56% vs Gemini -39.57%로 **수익 기여는 중립(약간 열위)**이고, 그래서 판단 두뇌는 끌 수 있는 옵션입니다. 반면 청구서 레이어는 고면 **막을 방법이 없는 공격 하나가 열립니다.** AI가 부속품인 자리와 대체 불가인 자리를 저희가 구분해 두었습니다.

'얼마까지'를 말하는 팀은 많습니다. '무엇에 대해'를 묻는 칸이 비어 있었습니다

저희가 소스·문서로 직접 확인한 범위의 세 곳과 나란히 놓습니다. 확인하지 못한 칸은 물음표로 남깁니다 — 없다고 적지 않습니다.

먼저, 겹치는 것
개인키 미접근 · 1회 서명 후 자율 실행 · 스코프 위임 · x402 레일 · 테스트넷 배포 — 이 다섯은 **경쟁군과 겹칩니다**. 차별점으로 쓰지 않습니다. 아래 네 줄만
저희 칸입니다.

비교 축	AGENTFABRIC Cronos x402 1위	CIRCLE AGENT WALLETS	KYVERN	402 GUARD
수취인 검사 — 누구에게	—	✓	✓	✓
금액 한도 집행 (표시가 아니라)	UI 표시용 *	?	?	✓ 서명 직전 차감
청구서 ↔ 합의 건적 대조	—	—	—	✓ 하드 8종 + 설명 의미 대조
매수·매도 양 레그 대칭	—	—	—	✓

* AgentFabric 은 자기 소스에 Advisory limits (NOT enforced on-chain, just for UI display) 라고 적어 두었습니다. **정직하게 단 플래그입니다** — 그들의 온체인 세션키는 '무엇을
컨트롤 수 있는가'까지 강제합니다. '얼마를-누구에게'가 아직 그 레이어에 없을 뿐입니다. · **저희 한도 집행도 온체인이 아닙니다** — 에이전트 프로세스 안의 코드가 서명 직전에 차감합니다
(ap2_mandate.py). 온체인 이관은 로드맵이고, **레일은 이미 실증했습니다** — SPL Token 위임으로 예산 상한을 체인이 집행하고 초과 결제를 거절하는 것을 독립 스크립트로 확인했습니다
(scripts/demo_delegation.py). **제품 경로에는 아직 연결하지 않았습니다.** · ? = 저희가 확인하지 못한 칸 — = **저희가 확인한 범위에서** 발견하지 못함. "세계 최초"라고 말하지 않습니다.

지금 도는 것과, 팔릴 것

현재 구현

거래 수수료 0.3%

브로커 에이전트가 건적에 수수료를 분리 표기하고, 누적 수수료를 화면에 표시합니다. **지금 코드에 있고 화면에 뜨는 유일한 단입입니다.**

에이전트 1대가 월 15,000 USDC 거래 → 월 45 USDC

제품화 1 · 단가 가정

게이트 SDK · 검사 1건 \$0.001

에이전트에 결제를 붙이는 개발사가 게이트를 라이브러리로 가져다 씁니다. 결제가 늘수록 검사도 늘어 건당 과금이 자연스럽습니다. **원가 근거** — 의미 대조는 결제 시도마다만 돌고 숫자를 가린 '서식' 캐시를 씁니다. 실제 모델 판정으로 돌린 실측이 **14건 대조 = LLM 1회**(캐시 13)였습니다.

제품화 2 · 단가 가정

정책·감사 구독 · 월 \$200 / \$2,000

여러 에이전트의 위임 한도를 한 곳에서 관리하고, 차단·유출·오탐 이력을 감사 가능한 형태로 남기는 관리 도구. 팀 월 \$200 · 기업 월 \$2,000.

12개월 목표 시나리오 - 전부 가정

총	목표	단가	월 매출
개발사 — 게이트 SDK	20곳 · 각 월 100만 검사	검사 1건 \$0.001	\$20,000
플랫폼 — 정책·감사 구독	기업 3곳	월 \$2,000	\$6,000
자율 매매 — 거래 수수료	에이전트 200대 · 대당 월 15,000 USDC	0.3%	\$9,000
합계			월 \$35,000

정직하게 — 위 표는 전부 가정입니다. 출처를 댈 수 없는 시장 규모(TAM) 수치는 적지 않았습니다. 검증된 것은 1단뿐이고(BROKER_FEE_BPS=30 이 코드에 있고 화면에 누적액이 뜹니다), 2·3단은 단가 가설입니다.

숨기지 않고 적습니다

지금의 한계

다섯 가지를 그대로 밝힙니다

- **수익률** 강한 상승장에서는 매수후보유를 이기지 못합니다(+4.90% 대 +30.19%). 저희는 알파를 팔지 않습니다.
- **자산** devnet에는 토큰화 주식이 존재하지 않아 자체 발행 토큰으로 시연합니다.
- **결제 통화** 초기 devnet 증빙의 'USDC' 는 구매자 지갑이 민트 권한을 쥔 테스트 토큰이었습니다. 지적당하기 전에 밝히고, **2026-07-31 Circle 공식 민트로 재실증했습니다** — 발행 권한은 저희 것이 아닙니다.
- **규격** 표준 402 응답은 **매수 레그**만입니다. 매도(환매)는 자산이 반대로 흐르는 구조라 A2A 메시지로 남겨 뒀고, 이 사실을 규격 문서에 미구현으로 명시해 공개합니다.
- **AI 기여** 수익 기여는 중립이며, 낙폭 축소에서만 유의미합니다.

다음

실물로 가는 경로는 짧습니다

- **실물 토큰화 주식** 메인넷에는 이미 존재합니다. 민트 상수와 스왑 경로 두 곳을 교체하면 같은 게이트가 실물 위에서 돕니다.
- **매도 레그 표준화** 매수는 이미 표준 402로 왕복합니다. 자산이 반대로 흐르는 환매도 같은 규격 위에 올리는 것이 다음 과제입니다.
- **온체인 예산 집행 배선** 한도를 체인이 강제하는 레일은 이미 실증했습니다(SPL 위임). 남은 것은 그 레일을 제품 결제 경로에 연결하는 일이고, 증빙 파일이 `wired_into_product: false` 로 그 경계를 표시하고 있습니다.
- **규제** 자산을 대신 보관하지 않는 자기수탁 도구 구조를 유지합니다. 토큰화 주식은 증권으로 취급되며 국내 제도는 2027년 시행 예정입니다.

직접 재현해 보십시오

기술 스택

구성

- 언어·런타임 Python 3.10 / 3.11
- AI Google Gemini API (무료 티어) — 판단·브리핑, 실패 시 규칙 대체
- 블록체인 Solana devnet · localnet, **Circle 공식 devnet USDC(SPL)**
— 주식만 자체 발행
- 프로토콜 자체 x402 정산 + AP2 위임 + A2A 협의 + **402 Guard**
- 웹 FastAPI + SSE, 외부 라이브러리 없는 정적 대시보드
- 인프라 Google Cloud Run, Firestore, Secret Manager

재현 절차

세 줄이면 확인됩니다

—저장소

```
git clone https://github.com/Loda1002/SynapStock
```

—표준 402 응답

```
curl -i -X POST \
https://synapstock-766888967498.asia-
northeast3.run.app/broker/orders \
-H 'Content-Type: application/json' \
-d '{"symbol": "tAAPL", "spend_usdc": "10", "price_usdc": "200"}'
```

—공격·차단 매트릭스

```
python -m scripts.red_team --report
```

—거부 4종 데모

```
python -m scripts.demo_rejections
```

—온체인 증빙

```
artifacts/tx/*.json → explorer ?cluster=devnet
```

모든 결제 실행은 트랜잭션 해시와 전후 잔액을 파일로 남깁니다.
화면의 숫자와 체인의 숫자가 일치하는지 직접 대조하실 수 있습니다.

402 Guard — x402 는 구매자의 **키**를 지킵니다. 무엇에 서명하는지는 **저희가** 지킵니다.